

NewBook/image-sources

NewBook/html

NewBook/html/images

NewBook/ebook

NewBook/ebook/images

- **Parameter expansion:** When bash executes a script, it creates these special variables for the script.

Shell variable	Use
\$0	Name of the shell script
\$1, \$2,...	Positional parameters or arguments passed to the script
\$#	Total count of arguments passed to the script
\$?	Exit status of last command
\$*	All arguments (double-quoted)
\$@	All arguments (individually double-quoted)
\$\$	Process ID of current shell/script

At the terminal, \$0 will usually expand to the shell program (/bin/bash).

On a terminal, you can use the *set* command to *ipso facto* specify parameters to the current shell.

```
# Displays 0
set $?
# Displays an empty string and causes a new line
echo $*
# Sets hello and world as parameters to current shell
set -- hello world
# Displays 2 (the number of parameters)
echo $#
# Displays world
echo $2
# Remove parameters to current shell
set --
# Displays 0 (as earlier)
set $?
```

The option -- represents the end of options and implies that whatever follows it must be command parameters.

- **Command substitution:** Instead of backquotes, you can use the form \$(commands) to capture the output of those commands for use in some other commands or variables. It makes quoting and escaping much easier.
- **Variable substitution:** You can use these substitutions with command parameters (created by bash for a shell

script) or with variables that you have created.

Substitution	Effect
\${var1:-var2}	If var1 is null or does not exist, var2 is used
\${var1:=var2}	If var1 is null or does not exist, value of var2 is used and set to var1
\${var1:?msg}	If var1 is null or does not exist, msg is displayed as error
\${var1:+var2}	If var1 exists, var2 is used but not set to var1
\${var:offset}	Everything of var after offset number of characters
\${var:offset:length}	Length number of characters of var after offset number of characters
\${!prefix*} \${!prefix@}	All variables names beginning with prefix
\${!var[@]} \${!var[*]}	All indexes of array variable var
\${#var}	Length of value of var
\${var#drop}	Value of var without prefix matching Regex pattern drop
\${var##drop}	Empty string if prefix matches Regex pattern drop
\${var%drop}	Value of var without suffix matching Regex pattern drop
\${var%%drop}	Empty string if suffix matches Regex pattern drop
\${var^letter}	Changes first letter of var to uppercase if it matches letter (any alphabet, * or ?) If letter is not specified, all first letter(s) of var will be changed to uppercase
\${var^^letter}	Changes any letter of var to uppercase if it matches letter (any alphabet, * or ?) If letter is not specified, all letter(s) of var will be changed to uppercase
\${var,letter}	Changes first letter of var to lowercase if it matches letter (any alphabet, * or ?) If letter is not specified, all first letter(s) of var will be changed to lowercase
\${var,,letter}	Changes any letter of var to lowercase if it matches letter (any alphabet, * or ?) If letter is not specified, all letter(s) of var will be changed to lowercase
\${var/find/replace}	Value of var with instances of <i>find</i> replaced with <i>replace</i> . If <i>find</i> begins with '#', then a match is made at the beginning. A '%' makes it match at the end.